Clustering

Тема 4.2: деревья решений
Трек: beginners

Фирсанова В.И.

НИУ ВШЭ

Что такое дерево решений?

Свойство 1: дерево решений — непараметрический алгоритм

Параметрические алгоритмы

- Предполагают, что данные подчиняются определенному распределению или закону, который описывается фиксированным набором параметров (коэффициентов).
- Примеры:
  - линейная регрессия (y = w*x + b),
  - логистическая регрессия,
  - нейросети (веса и bias'ы).
- Количество параметров фиксировано и не зависит от объема обучающих данных (оно задается архитектурой до обучения).

Что такое дерево решений?

Свойство 1: дерево решений — непараметрический алгоритм

Непараметрические алгоритмы

- Не выдвигают предположений о форме (напр., выпуклость) функции и количестве параметров кроме принципа локальности.
- Принцип локальности: близкие точки в векторном пространстве признаков указывают на сходные классы. Когда мы работаем с этим принципом:
  - деревья решений,
  - k-средних,
  - SVM с RBF.
- Количество параметров не фиксировано до начала обучения и растёт с увеличением объема обучающих данных.

Что такое дерево решений?

Свойство 2: дерево решений может применяться как для классификации, так и для кластеризации.
Свойство 3: дерево решений имеет иерархическую древовидную структуру, состоящую из корневого узла, ветвей, внутренних узлов и листовых узлов.

Структуры элементы алгоритма

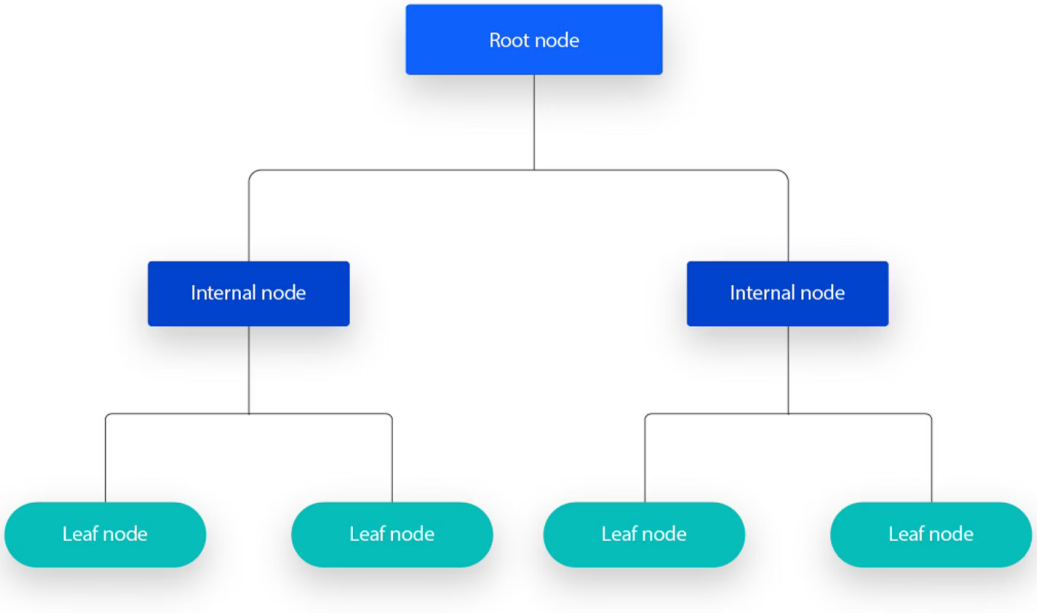

Рис.: Дерево решений начинается с корневого узла, который не имеет входящих ветвей. Исходящие ветви от корневого узла затем поступают во внутренние узлы (internal / decision nodes). Оба (и корневой, и внутренние) типа узлов проводят оценку для формирования однородных подмножеств (leaf / terminal nodes). Листовые узлы представляют все возможные результаты для набора данных.

Задача: бинарная классификация (одобрить кредит или нет?)

Входные данные (признаки):

- Возраст (лет)
- Доход (тыс. руб./мес)
- Есть ли работа? (Да/Нет)
- Есть ли недвижимость? (Да/Нет)

Цель (целевая переменная):

- Кредит (Одобрен / Отказан)

Датасет

Клиент	Возраст	Доход (т. р.)	Работа	Недвиж.	Кредит
1	45	120	Да	Да	Одобрен
2	38	80	Да	Нет	Одобрен
3	25	40	Да	Нет	Отказан
4	31	60	Нет	Да	Отказан
5	52	150	Да	Да	Одобрен
6	22	25	Нет	Нет	Отказан
7	29	45	Да	Нет	Отказан
8	41	95	Да	Да	Одобрен

Построение дерева. Этап 1

Выбираем корневой узел: алгоритм ищет правило (вопрос), которое лучше всего разделит клиентов на "Одобренных"и "Отказанных"(перебирает все признаки и выбирает тот, который минимизирует неопределенность в дочерних узлах, используя жадный алгоритм).
Для этого нужно измерить неопределенность:

- Энтропия
- Критерий Джини (Gini Impurity)

Формула Джини

$$G = 1 - \sum_{i=1}^{C} p_i^2$$

где p_i — доля класса i в узле (количество сэмплов на 1 класс), C — количество классов (здесь: количество клиентов).

Для корневого узла:

- $p_{\text{Одобрен}} = \frac{4}{8} = 0.5$
- $p_{\text{Отказан}} = \frac{4}{8} = 0.5$

$$G_{\text{корень}} = 1 - (0.5^2 + 0.5^2) = 1 - (0.25 + 0.25) = 1 - 0.5 = 0.5$$

Что значит G = 0.5?

G = 0.5 в корне — это не про выбор вопроса, а точка отсчета. G (примесь Джини) показывает, насколько неопределенным является узел до любого разделения.

- G = 0, узел идеально чистый (все объекты одного класса).
- G = 0.5 (максимум для двух классов), максимальная неопределенность (классы разделены поровну).

В нашем случае G = 0.5 означает, что сейчас в корневом узле полный хаос — 4 одобренных и 4 отказанных. Угадать класс нового клиента нельзя лучше, чем подбрасывая монетку.

Как использовать меру G?

Когда мы начнём перебирать разные вопросы для корневого узла (напр., Доход, Возраст, Работа и т.д.), мы будем вычислять:

$$\text{Выигрыш} = G_{\text{корень}} - G_{\text{после разделения}}$$

- Если $G_{\text{после разделения}} = 0.2$, то выигрыш = 0.5 - 0.2 = 0.3 (хорошо).
- Если $G_{\text{после разделения}} = 0$, то выигрыш = 0.5 - 0 = 0.5 (идеально).
- Если $G_{\text{после разделения}} = 0.5$, то выигрыш = 0.5 - 0.5 = 0 (бесполезный вопрос).

Перебираем все возможные вопросы

Для каждого признака алгоритм перебирает все возможные пороги (для числовых) или значения (для категориальных).

Признак Доход (числовой)

1. Сортируем уникальные значения [25, 40, 45, 60, 80, 95, 120, 150].
2. Исследуем пороги (обычно — среднее между соседними значениями).

Пороги

Порог	Од. слева	От. слева	Од. справа	От. справа
32.5 (м. 25 и 40)	0	1	4	3
42.5 (м. 40 и 45)	0	2	4	2
52.5 (м. 45 и 60)	1	2	3	2
52.5 (м. 45 и 60)	1	2	3	2
70 (м. 60 и 80)	2	2	2	2
87.5 (м. 80 и 95)	3	2	1	2
107.5 (м. 95 и 120)	3	3	1	1
135 (м. 120 и 150)	4	3	0	1

Оцениваем качество каждого вопроса

Для каждого кандидата считаем взвешенную сумму примесей дочерних узлов:

$$G_{\text{после}} = \frac{n_{\text{левый}}}{n_{\text{всего}}} \cdot G_{\text{левый}} + \frac{n_{\text{правый}}}{n_{\text{всего}}} \cdot G_{\text{правый}}$$

где:

- $n_{\text{левый}}$ — количество объектов в левом дочернем узле
- $n_{\text{правый}}$ — количество объектов в правом дочернем узле
- $n_{\text{всего}}$ — количество объектов в родительском узле
- $G_{\text{левый}}$ — примесь (Gini / энтропия) левого узла
- $G_{\text{правый}}$ — примесь правого узла

Как используется взмешенная сумма примесей

Считаем выигрыш информации (Information Gain):

$$\mathrm{IG} = G_{\text{до}} - G_{\text{после}}$$

Выбираем вопрос с максимальным IG.

Пример для порога Доход ≥ 70

Левый узел (Доход < 70): клиенты 3(40),4(60),6(25),7(45) $\rightarrow 2, 2$

$$G_{\text{левый}} = 1 - (0.5^2 + 0.5^2) = 0.5$$

Правый узел (Доход ≥ 70) : $1(120), 2(80), 5(150), 8(95) \rightarrow 2, 2$

$$G_{\text{правый}} = 1 - (0.5^2 + 0.5^2) = 0.5$$

$$G_{\text{после}} = \frac{4}{8} \cdot 0.5 + \frac{4}{8} \cdot 0.5 = 0.25 + 0.25 = 0.5$$

$$\text{IG} = 0.5 - 0.5 = 0$$

Выигрыша нет

Рассчитаем для порога Доход ≥ 50

Левый узел (Доход ≥ 50) : $1, 2, 4, 5, 8 \to 4, 1$

$$p_{\text{Од}} = \frac{4}{5} = 0.8, \quad p_{\text{От}} = \frac{1}{5} = 0.2$$

$$G_{\text{левый}} = 1 - (0.8^2 + 0.2^2) = 1 - (0.64 + 0.04) = 0.32$$

Правый узел (Доход < 50): Клиенты 3,6,7 $\to 0, 3$

$$p_{\text{Од}} = 0, \quad p_{\text{От}} = 1$$

$$G_{\text{правый}} = 1 - (0^2 + 1^2) = 1 - 1 = 0$$

$$G_{\text{после}} = \frac{5}{8} \cdot 0.32 + \frac{3}{8} \cdot 0 = 0.2 + 0 = 0.2$$

$$\text{IG} = 0.5 - 0.2 = 0.3$$

Выигрыш 0.3

Признак Работа (бинарный)

- Работа = Да: Клиенты 1,2,3,5,7,8 → 4 Одобрена, 2 Отказана (G = 1 - ((4/6)² + (2/6)²) = 1 - (0.444 + 0.111) = 0.445)
- Работа = Нет: Клиенты 4,6 → 0 Одобрена, 2 Отказана (G = 0)

$$G_{\text{после}} = \frac{6}{8} \cdot 0.445 + \frac{2}{8} \cdot 0 = 0.334$$

$$\text{IG} = 0.5 - 0.334 = 0.166$$

Хуже, чем порог 50 по доходу (0.3)

Признак Недвижимость (бинарный)

- Недвижимость = Да: Клиенты 1,4,5,8 $\rightarrow$ 3 Одобрена, 1 Отказан (G = 1 - ((3/4)^2 + (1/4)^2) = 1 - (0.5625 + 0.0625) = 0.375)
- Недвижимость = Нет: Клиенты 2,3,6,7 $\rightarrow$ 1 Одобрен, 3 Отказана (G = 1 - ((1/4)^2 + (3/4)^2) = 1 - (0.0625 + 0.5625) = 0.375)

$$G_{\text{после}} = \frac{4}{8} \cdot 0.375 + \frac{4}{8} \cdot 0.375 = 0.375$$

$$\text{IG} = 0.5 - 0.375 = 0.125$$

Самый плохой вариант

Признак Возраст (числовой)

Перебираем пороги. Лучший, предположим, Возраст ≥ 33.5 :

- Возраст ≥ 33.5 : $1(45), 2(38), 5(52), 8(41) \to 4, 0$
- Возраст < 33.5: 3(25),4(31),6(22),7(29) $\to 0, 4$.

2 чистых узла; $G_{\text{после}} = 0$, IG $= 0.5$
Возраст даёт максимальнный IG $= 0.5$

Узлы решений

Определение

В узлах решений (внутренние узлы, decision nodes) происходит перебор вопросов.

Узел решения — любой узел, у которого есть дочерние узлы (не лист). Он содержит:

- Вопрос (условие) — например, Доход 50
- Две или более ветвей (в бинарном дереве — две)

Алгоритм работает рекурсивно

```
function build_tree(data):
    if should_stop(data):
        return Leaf(prediction)
    else:
        best_question = find_best_question(data)  перебор вопросов
        left_data, right_data = split(data, best_question)

        left_child = build_tree(left_data)  рекурсивный вызов
        right_child = build_tree(right_data)  рекурсивный вызов
        return DecisionNode(best_question, left_child, right_child)
```

Каждый узел решения выполняет свой собственный перебор вопросов только для своих данных.

QA

- Перебор происходит с использованием энтропии (или Джини).
  1. Делим данные по вопросу.
  2. Вычисляем взвешенную сумму примесей.
  3. Вычисляем IG.
  4. Выбираем вопрос с максимальным IG.
- В бинарном дереве данные делятся по порогу (числовое значение или категории "да-нет")

Терминальные узлы

Лист — это терминальный (конечный) узел, в который алгоритм переходит после перебора всех признаков.

Узел объявлен листом, когда "закончились вопросы". Алгоритм:

1. Останавливается для этой ветки
2. Вычисляет предсказание (моду для классификации, среднее для регрессии)

Is there a swell?
Yes
No
High wind
Low to no wind
Don't surf
Onshore wind direction
Offshore wind direction
Surf
Don't surf
Surf

Листовые узлы

Если корневой узел выбирает лучший вопрос для разделения выборки, то листовые используются для определения момента остановки поиска ответа на вопрос (принятия решения о предсказании).

В листовом узле алгоритм смотрит на моду (для классификации) или среднее (для регрессии):

- Лист: [Одобрен, Одобрен, Отказан, Одобрен]
- Предсказание = Одобрен (мода)

ID3 (Iterative Dichotomiser 3)

Ross Quinlan, 1986. Разработан на основе концепции «обучающихся деревьев» Ханта (1966) и теории информации Шеннона (энтропия).

Алгоритм рекурсивно выбирает признак, дающий максимальное уменьшение неопределенности (энтропии), и разбивает данные по всем его возможным значениям, строя дерево классификации.

$$IG(D, A) = H(D) - \sum_{v \in Values(A)} \frac{|D_v|}{|D|} \cdot H(D_v)$$

где $H(D) = -\sum_{i=1}^{C} p_i \log_2 p_i$ — энтропия Шеннона.

ID3 — Энтропия Шеннона

$$H(D) = -\sum_{i=1}^{C} p_i \log_2 p_i$$

где:

- D — множество объектов в узле
- C — количество классов
- p_i — доля объектов класса i в узле: $\frac{\text{count}(class_i)}{|D|}$
- $\log_2$ — логарифм по основанию 2 (измеряет информацию в битах)

Свойства энтропии:

- $H(D) = 0$ — все объекты одного класса (чистый узел)
- $H(D) = 1$ (для 2 классов) — классы поровну (максимальный хаос)

ID3 — Прирост информации (Information Gain)

$$IG(D, A) = H(D) - \sum_{v \in Values(A)} \frac{|D_v|}{|D|} \cdot H(D_v)$$

где:

- A — признак, по которому делаем разделение
- $Values(A)$ — все возможные значения признака A
- D_v — подмножество объектов, у которых признак A принял значение v
- $|D_v|$ — количество объектов в подмножестве v
- $|D|$ — общее количество объектов

Считает, насколько уменьшилась неопределенность после разделения по признаку A.
Что выбирает ID3: признак A с максимальным $IG(D, A)$.

C4.5

Ross Quinlan, 1993. Улучшенная версия ID3, созданная для преодоления его ограничений: работа с числовыми признаками, пропусками в данных и переобучением.

Алгоритм строит дерево классификации, используя либо Information Gain, либо нормализованный Gain Ratio, и выполняет последующую обрезку ветвей для повышения обобщающей способности.

$$\text{GainRatio}(D, A) = \frac{IG(D, A)}{\text{SplitInfo}(D, A)}$$

где $\text{SplitInfo}(D, A) = -\sum_v \frac{|D_v|}{|D|} \log_2 \frac{|D_v|}{|D|}$ — энтропия разбиения.

C4.5 — Gain Ratio (коэффициент прироста)

IG отдает предпочтение признакам с большим количеством значений (например, ID клиента). Split Information (информация о разбиении) используется для нормализации.

$$\text{SplitInfo}(D, A) = -\sum_{v \in Values(A)} \frac{|D_v|}{|D|} \log_2 \frac{|D_v|}{|D|}$$

Это энтропия самого разбиения, не целевой переменной. Она растет, когда у признака много значений.

$$\text{GainRatio}(D, A) = \frac{IG(D, A)}{\text{SplitInfo}(D, A)}$$

Прирост информации, нормированный на информацию о разбиении. Штрафует признаки с большим количеством значений.

CART (Classification and Regression Trees)

Leo Breiman, Jerome Friedman, Richard Olshen, Charles Stone, 1984. Разработан в Калифорнийском университете как универсальный метод статистического обучения, объединяющий классификацию и регрессию в единой бинарной древовидной структуре.

Алгоритм рекурсивно подбирает бинарное разделение данных по одному признаку, минимизирующее примесь Джини (для классификации) или среднеквадратичную ошибку (для регрессии). Выбор вопроса основан на переборе всех признаков и порогов, цель — максимизировать чистоту в дочерних узлах.

CART

Что считает алгоритм:

$$G = 1 - \sum_{i=1}^{C} p_i^2$$

где p_i — доля класса i в узле.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \bar{y})^2$$

где $\bar{y}$ — среднее значение целевой переменной в узле.

Что читать

- https://www.ibm.com/think/topics/decision-trees
- https://www.geeksforgeeks.org/machine-learning/tree-based-machine-learning-algorithms/
- https://arxiv.org/pdf/1407.7502
- https://vdsbook.com/12-rf